Nathaniel Wooley 2/22/22

G01017132

ECE 621 Project 1

$$G(z) = \frac{z^{-1}(9 + 6z^{-1} + 6z^{-2})}{(1 - 0.8z^{-1})(1 - 0.68z^{-1})(1 - 0.5z^{-1})} = \frac{z^{-1}(9 + 6z^{-1} + 6z^{-2})}{1 - 1.98z^{-1} + 1.284z^{-2} + 0.272z^{-3}}$$

In this project, we were given a transfer function with some define parameters and some undefined parameters. The first three letters of my family name are W, O, O. This gave me the parameters 9, 6 and 6 which are in the system defined above.

Question 1: Simulate the Above System

With this system defined, I built the system in MATLAB with the 'filt' function. The 'filt' function is part of MATLAB's Control Systems Toolbox and uses a numerator and a denominator to specify a discrete transfer function in the Z-transform format. This was a very useful function to create my system quickly and effectively in MATLAB. The code and output of this function from the MATLAB console is shown below.

Code:

```
%1. Create the transfer function and simulate the system
num = [0 9 6 6];
den = [1 -1.98 1.284 -0.272];
sys = filt(num, den)
```

Output:

```
sys =

          9 z^-1 + 6 z^-2 + 6 z^-3
  -----------------------------------------
  1 - 1.98 z^-1 + 1.284 z^-2 - 0.272 z^-3
```

As shown above, the system is correctly represented in MATLAB based on the calculations shown at the top of this page.

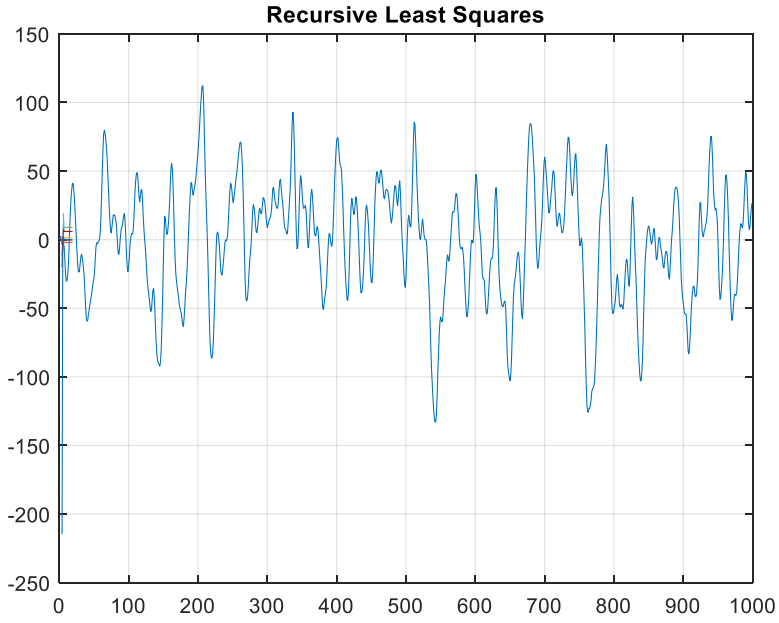
Question 2: Run the System with Zero-Mean Random Input

Once the system was defined in MATLAB, I used the 'lsim' function to simulate the above system with a zero-mean random input. The zero mean random input is generated using the MATLAB 'rand' function. This function generates a set of 1000 random values with a mean of approximately zero based on the adjustment of subtracting 0.5 from each value. Below is the console output of the calculated mean of the input as well as the code to generate the input and simulate the system.

```
%2. Simulate the system with zero mean random input (1000 samples)
T = 1;
t = 0:T:999;
n = length(t)

input = rand(n,1) - 0.5;
output = lsim(sys, input, t);
input_mean = mean(input)
variance = var(output)
```

```
input_mean =     variance =

    0.0023          2.3137e+03
```

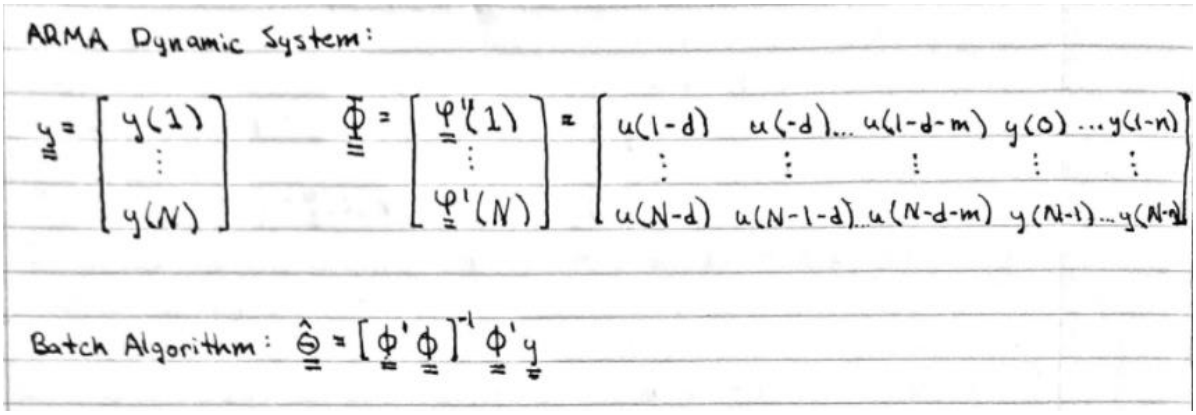
Next, the lsim function was used (shown above) to simulate the system with this zero-mean random input. The 'lsim' function is a function that simulates the response of a dynamic system to an arbitrary input. Below is the output of system based on the zero-mean random input.



During this, the variance of the output is also computed (shown above). The output variance is usually in the range of 2e+3 but can change based on the randomly generated input.

Question 3: Batch Least Squares Algorithm

After running the system with 1000 samples, I wrote a function for calculating the Batch Least Squares to identify the parameters. For the Batch Least Squares algorithm, I defined the y and Φ matrices as shown below. The y matrix is the set of outputs to the system and the Φ matrix is a set of the φ regressors. These regressors are then further defined by the inputs and output of the system. Below are the equations used to define y, Φ and φ.



The code for these equations can be found in BatchLeastSquares.m. This function takes the input, output, and time vector of the system, calculates phi for this and then estimates the parameters using the phi matrix. It is worth noting that the code has rows and columns flipped for phi as I decided at the beginning of the project that this method would be easier in MATLAB for some of the calculations and for graphing in MATLAB. The code for this function is shown below.

```
function [ theta ] = BatchLeastSquares ( input, output, time)
% 3 Batch LS program to identify the parameters
numSamples = length(time);

% Create the phi matrix based
for i=3:numSamples
    for j=1:3
        if(i-j <=0)
            phi(i,j)=0;
        else
            phi(i,j)=-output(i-j);
        end
    end
    for j=0:3
        if(i-j-1 <=0)
            phi(i,j+4)=0;
        else
            phi(i,j+4)=input(i-j-1);
        end
    end
end

% Calculate Batch Parameters
theta =inv(phi' * phi)*phi'*output;
```

Question 4: Batch Parameter Estimation

After creating the program for the batch LS algorithm, I used this function to drive the simulated system with my random input and identify the parameters. This was done by calling the function I created with the following line of code.

```
% 4 Batch LS Function and drive system to identify parameters
theta_bls = BatchLeastSquares(input, output, t);
```

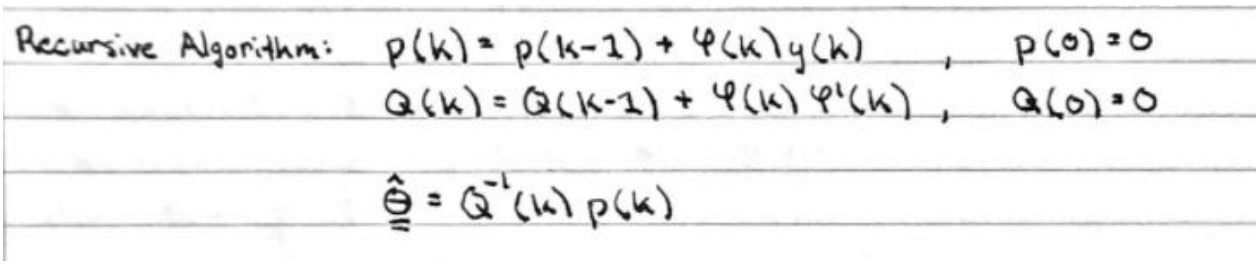
The BLS function returns a matrix containing the batch estimated parameters, which is pictured below for the run simulation.

-1.9800
1.2840
-0.2720
9.0000
6.0000
6.0000
5.9459e-11

As shown, the parameters for b0, b1, & b2 match with the transfer function, as do the parameters a0, a1, a2 and a3.

Question 5: Recursive Least Squares Algorithm

After finishing the BLS algorithm, I moved onto creating a recursive LS function. This algorithm uses the same y, Φ and φ as defined above, with the key difference of an estimation occurring at each new sample which yields new y, Φ and φ matrices. The p, Q and theta calculations are shown below for this algorithm.



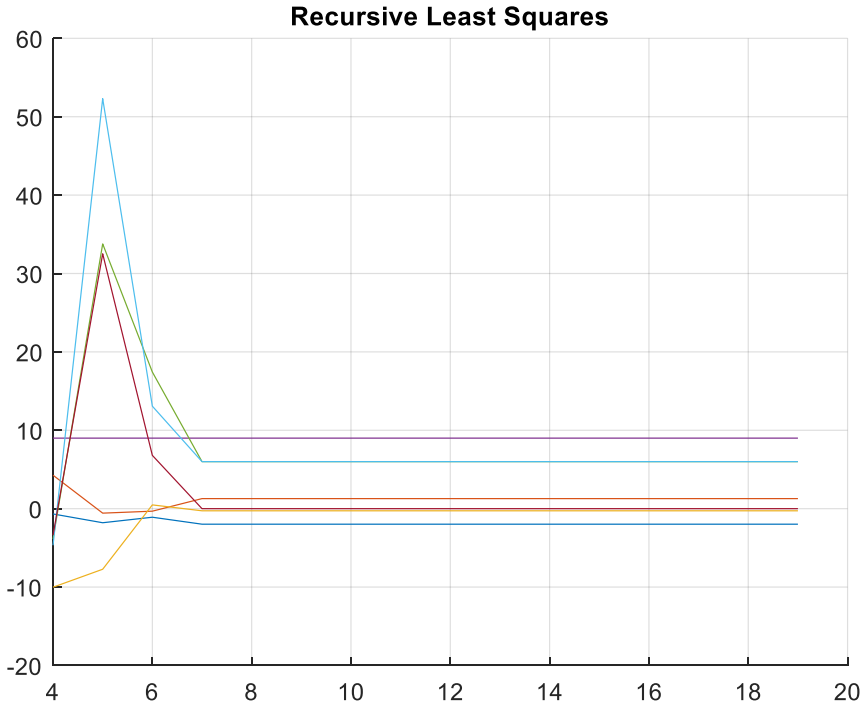
The p, Q and theta matrices are calculated on each iteration and have a starting value of 0. This leads to the estimation of the parameters changing slightly on each new sample based on current and previous data.

This function to run this algorithm lives in RecursiveLeastSquares.m, the function takes in the input, output, time vector of the system, and a graph counter. This function then takes these inputs to create the phi matrix, calculates P and Q and then recursively calculates theta for

each new sample. The code for this algorithm portion function can be found below, there also exists a graphing section of this function to create the graphs associated with this function.

```
for i=1:numSamples
    %Create the phi matrix
    for j=1:3
        if(i-j <=0)
            phi(i,j)=0;
        else
            phi(i,j)=-output(i-j);
        end
    end
    for j=0:3
        if(i-j-1 <=0)
            phi(i,j+4)=0;
        else
            phi(i,j+4)=input(i-j-1);
        end
    end

    % Recursively find theta
    p = p + phi(i,:)' * output(i);
    q = q + phi(i,:)' * phi (i,:);
    theta(:,i) = inv(q) * p;
end
```

The function for the RLS algorithm outputs a 7x1000 matrix of the estimations of the parameters as the simulation runs. Below is a graph of those 7 estimated parameters over time that is generated by the RLS function. The final estimated parameters and graph of those parameters is shown below.

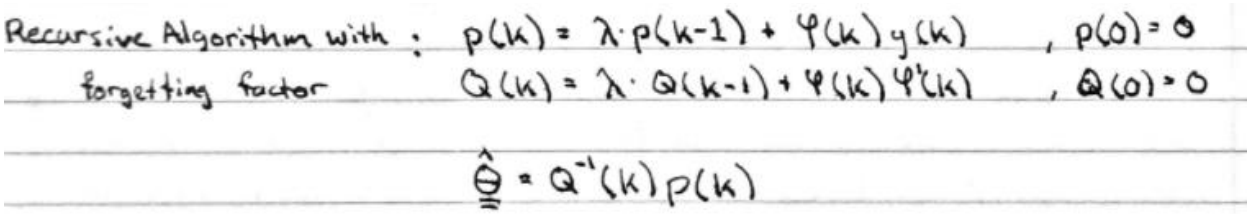


-1.9800
1.2840
-0.2720
9.0000
6.0000
6.0000
-1.6716e-10

The RLS algorith estimates the parameters same as the BLS algorithm, with the exception of the final parameter which is effectively 0 in both cases. As shown, the parameter estimates converge within the first ten estimates and stay at those values for the remainder of the simulation. The MATLAB code will show a graph of all 1000 samples if desired.

Question 6: Recursive Least Squares with Forgetting Factor & Change at t=411

With the basic recursive version of the LS algorithm, I added the forgetting factor to the RLS algorithm. This uses the same y, Φ and φ matrices as the basic recursive LS algorithm, but p and Q are defined with the inclusion of the forgetting factor. This is shown below.

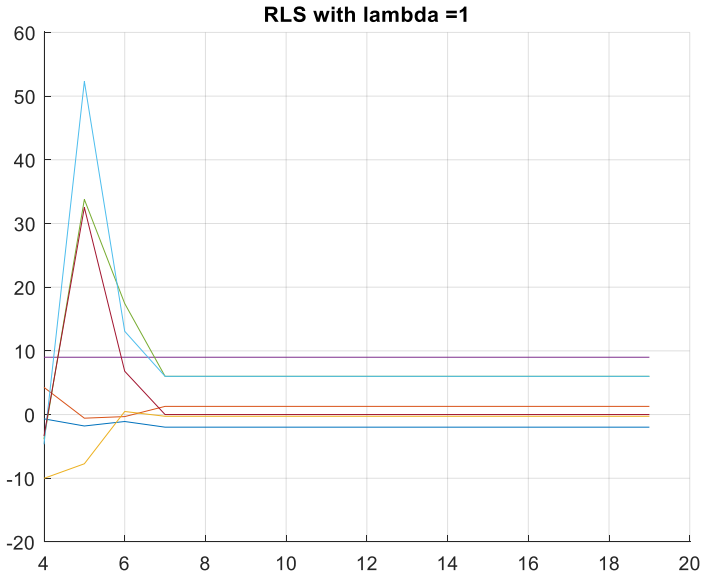


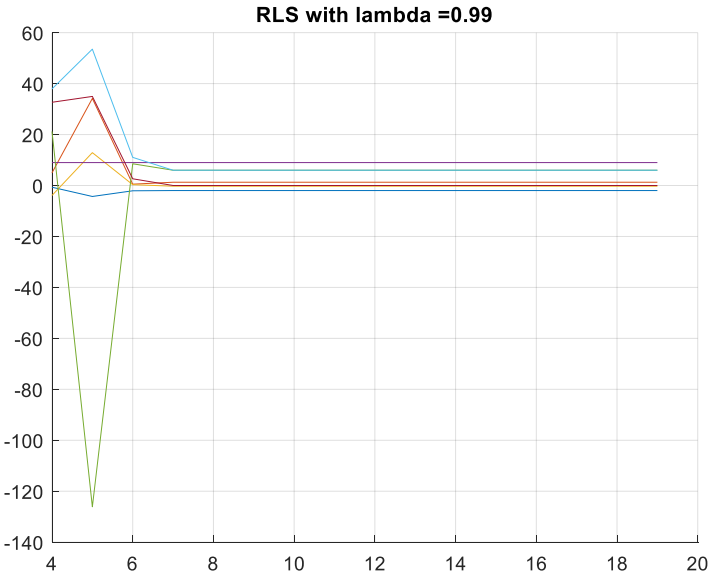
The function to define this algorithm is in RLS_forgetting.m and operates similar to the RLS function. The function takes in the input, output, changed output, lambda values, time vector of the system, time of change, and a graph counter. This function then creates the phi matrix, calculates P and Q, and then recursively calculates theta. The function outputs a 7x1000 matrix for each value of lambda, all stored in one cell matrix. The code for this function is shown below.

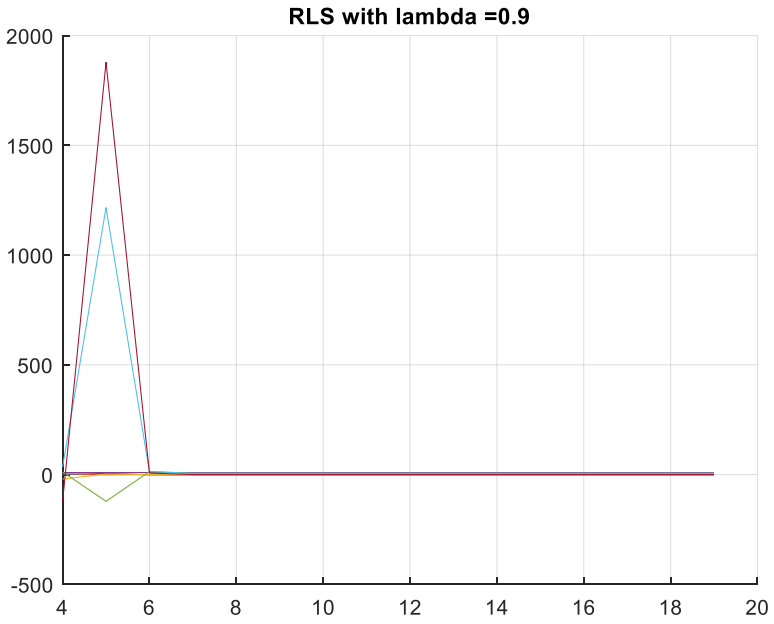
```
for(L = 1:length(lambda))
    % Reset output to original output
    output = original_output;
    p=0;
    q=0;
    for i=1:numSamples
        %Create the phi matrix
        for j=1:3
            if(i-j <=0)
                phi(i,j)=0;
            else
                phi(i,j)=-output(i-j);
            end
        end
        for j=0:3
            if(i-j-1 <=0)
                phi(i,j+4)=0;
            else
                phi(i,j+4)=input(i-j-1);
            end
        end

        % Recursively find theta
        p = lambda(L) * p + phi(i,:)' * output(i);
        q = lambda(L) * q + phi(i,:)' * phi (i,:);
        theta{L}(:,i) = inv(q) * p;

        % Change parameters
        if(i == changeParam)
            output = output2;
        end

    end
end
```

This code is able to successfully estimate the parameters for all given values of lambda. Below we can see the parameter estimation throughout the runtime of the system for each lambda value. The sudden change occurs at t=411 and there are multiple graphs shown for each run for better visibility. For this run and all following runs, the parameter changed is the b1 parameter, which has changed from 6 to 2 for the initial set of parameters to the second set of parameters.

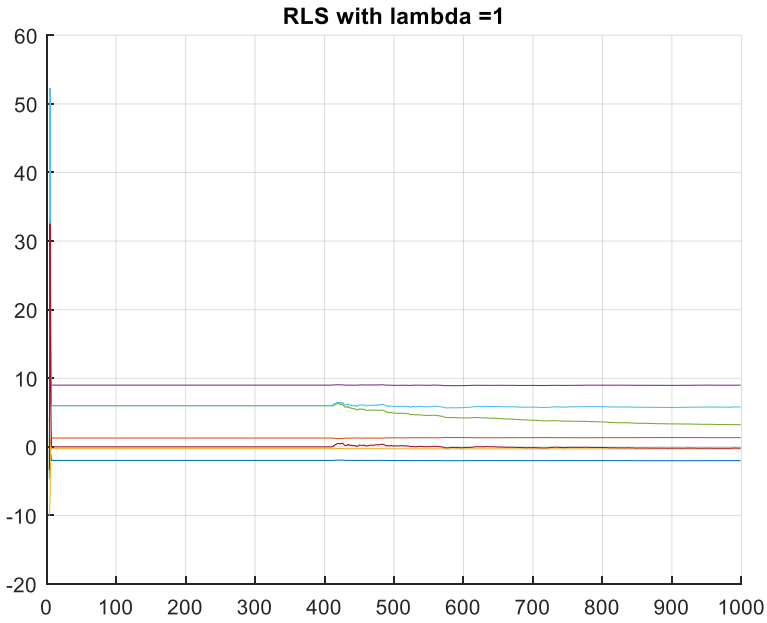
Below are each of the three lambda values for the first 20 iterations of the simulation as the initial parameters converge.

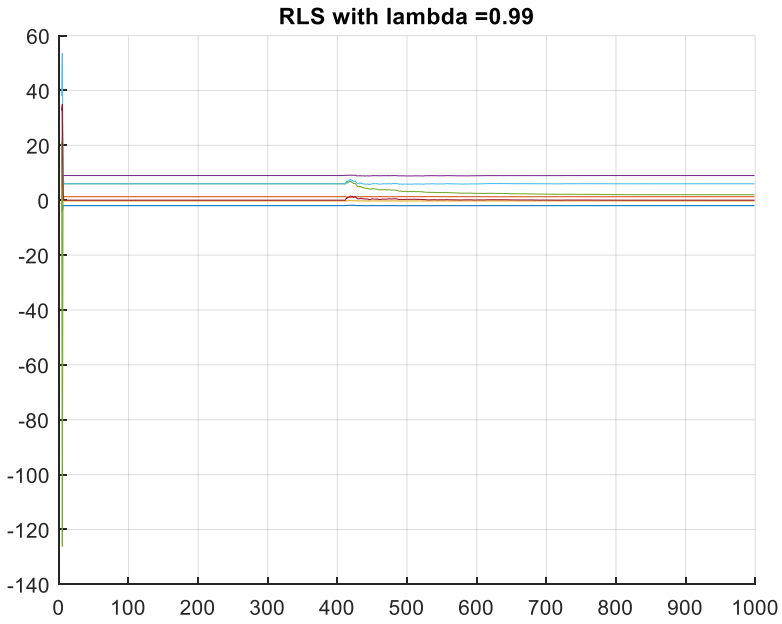




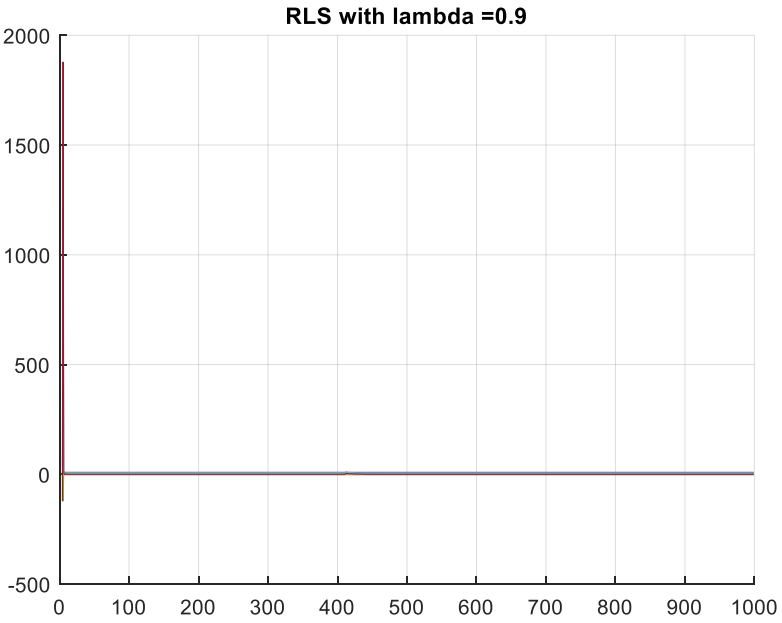


As the parameters quickly converge, they stay consistently at their values until the point of sudden change (t = 411). This can be seen in the following charts of the same simulation that runs until t=1000.





As the initial simulation doesn't show the sudden change of parameters for lambda = 0.9, a second graph has been added from another run of the simulation to show the sudden change in the simulation.



As can be seen in the charts above, the parameters experience changes at t=411 and the green line shifts from 6 to 2, which reflects the change in parameters in the two outputs. It can also be seen that the lower the lambda value, the faster the parameter estimation changes with lambda = 1 having the most gradual change and lambda = 0.9 having the most rapid change.

Question 7: Recursive Least squares with Forgetting Factor, Noise and Change at t=411

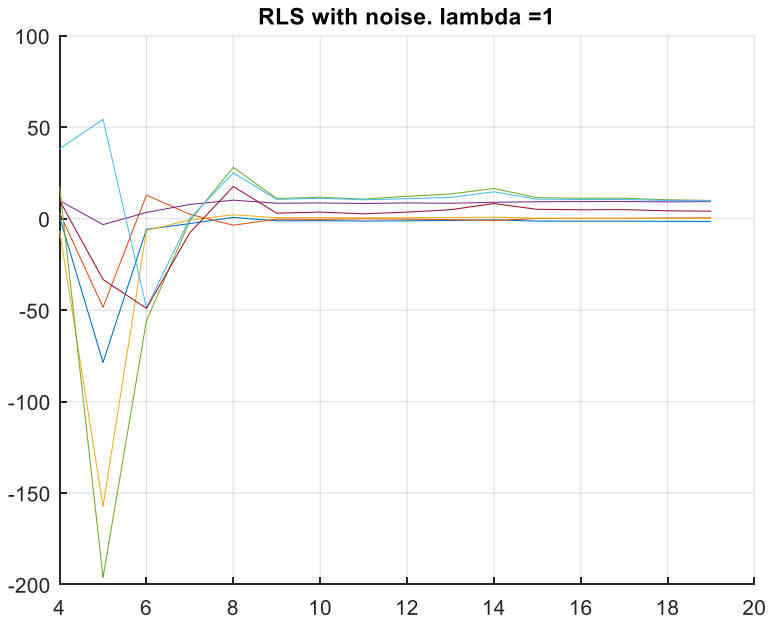
Next, I moved onto the next iteration of the RLS algorithm. This iteration has the forgetting factor, a sudden change and zero-mean noise to the output. The noise is generated in a similarly random way as the input but is limited to a standard deviation of approximately 1% of the standard deviation of the output. The noise was generated similar to the input, by utilizing the 'rand' function within MATLAB. This makes the noise random and independent of the input. The standard deviation of both the output and noise signals are shown below.

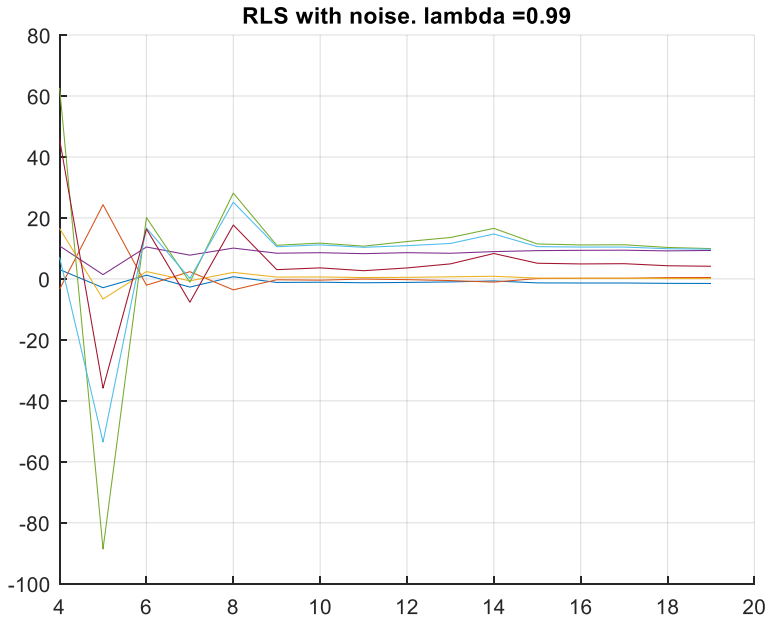
output_std =	noise_std =
47.4116	0.4882

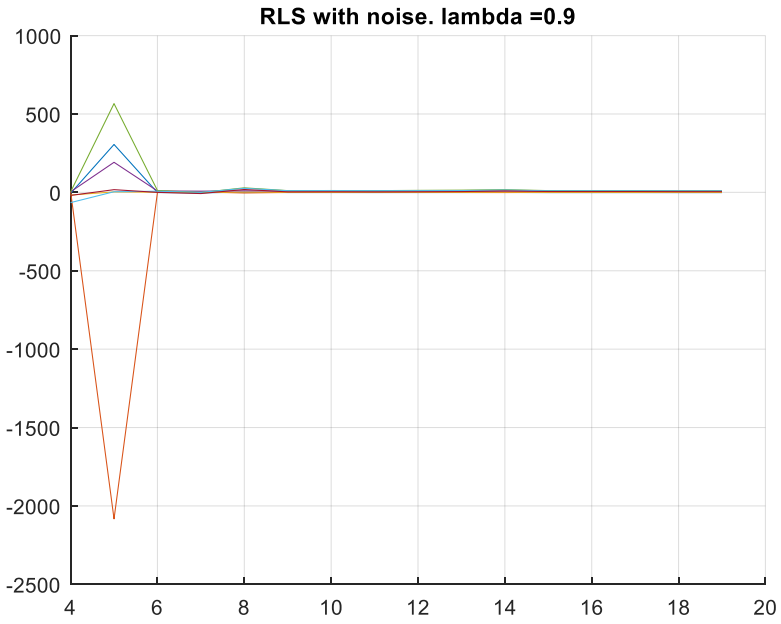
This function to define this algorithm is in RLS_noise.m and takes the input, output, noise, changed parameters output, lambda, time of change, time vector and a graph counter as inputs to the function. Then the function returns the estimated parameters over time for the given input and output and graphs these parameters over time. Each matrix of parameter estimations lives in a cell that becomes a part of a larger matrix for output of the whole function. The code for this algorithm is shown below.

```
numSamples = length(time);
original_output = output + noise;
for(L = 1:length(lambda))
    % Before each iteration with each new lamda, rebuild the system to its
    % original state so that the sudden change occurs newly on each run
    output = original_output;
    p=0;
    q=0;

    for i=1:numSamples
        %Create the phi matrix
        for j=1:3
            if(i-j <=0)
                phi(i,j)=0;
            else
                phi(i,j)=-output(i-j);
            end
        end
        for j=0:3
            if(i-j-1 <=0)
                phi(i,j+4)=0;
            else
                phi(i,j+4)=input(i-j-1);
            end
        end

        % Recursive calculation
        p = lambda(L) * p + phi(i,:)' * output(i);
        q = lambda(L) * q + phi(i,:)' * phi (i,:);
        theta{L}(:,i) = inv(q) * p;

        % Change parameters
        if(i == changeParam)
            output = output2;
        end

    end
end
```

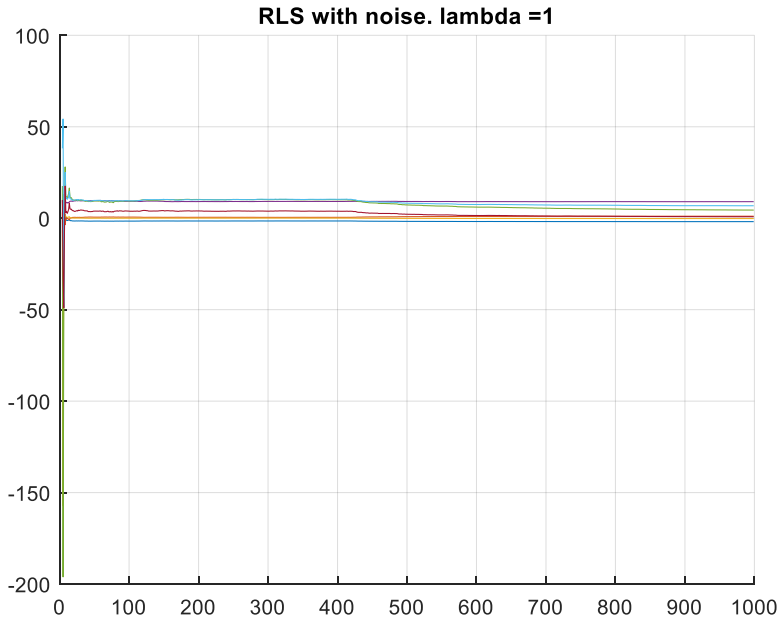
During this simulation, the graphs are based on the parameters of the original input up to t=411 and then the parameter estimation begins working with the second output that is based on the changed parameters and estimates those parameters up until t=1000. Below are the graphs for the initial convergence of these parameters up to t=20.

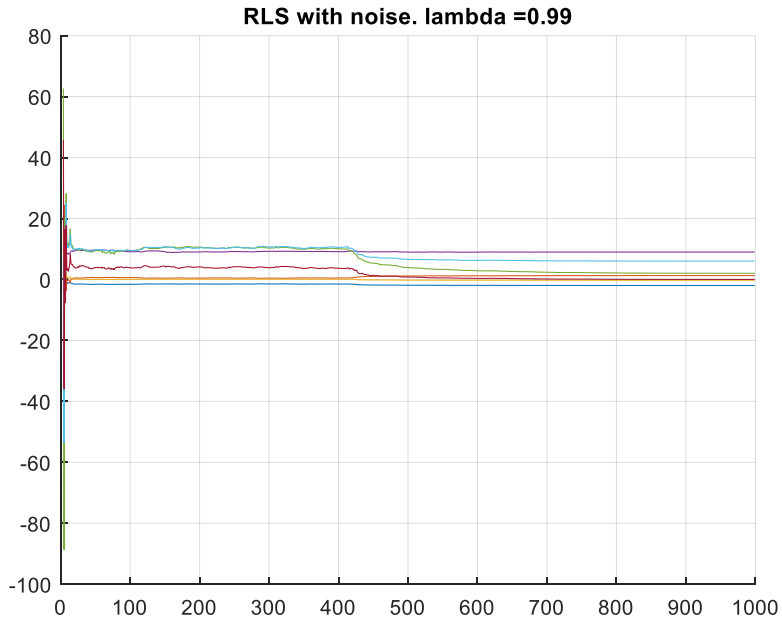


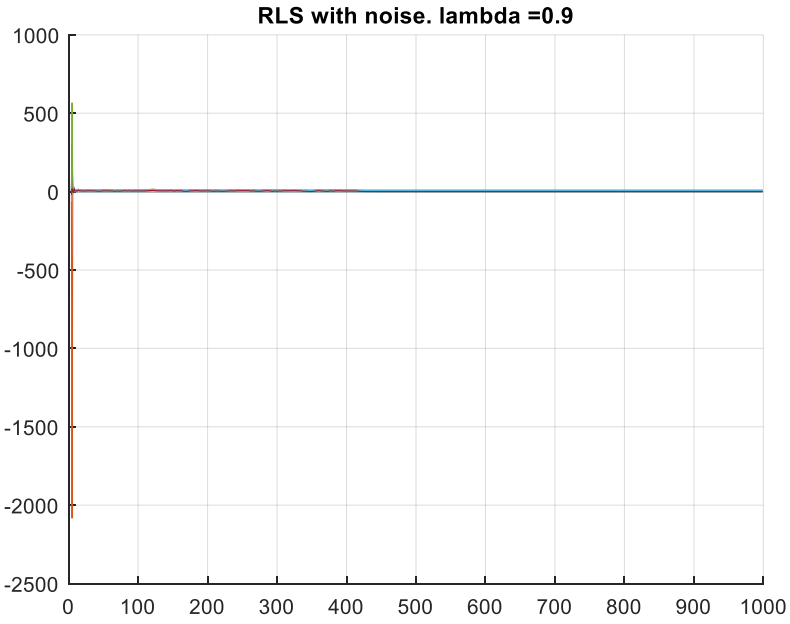




As the parameters quickly converge, they stay consistently at their values until the point of sudden change (t = 411). This can be seen in the following charts of the same simulation that runs until t=1000.



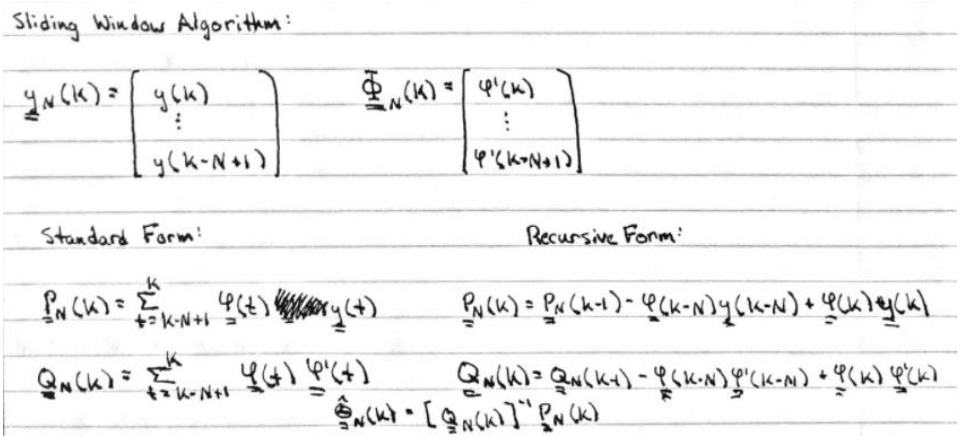




All three algorithms are able to accurately estimate the parameters early in the simulation and hold the estimation close to the actual value of the parameters despite the noise. All three lambda values are able to closely follow the parameters even through the sudden change. It is difficult to see this change of parameters on the graph with lambda = 0.9 due to the large initial spike in estimation, but the output matrices show that the parameters change accurately based on the change.

Question 8: Sliding Window Least Squares Algorithm

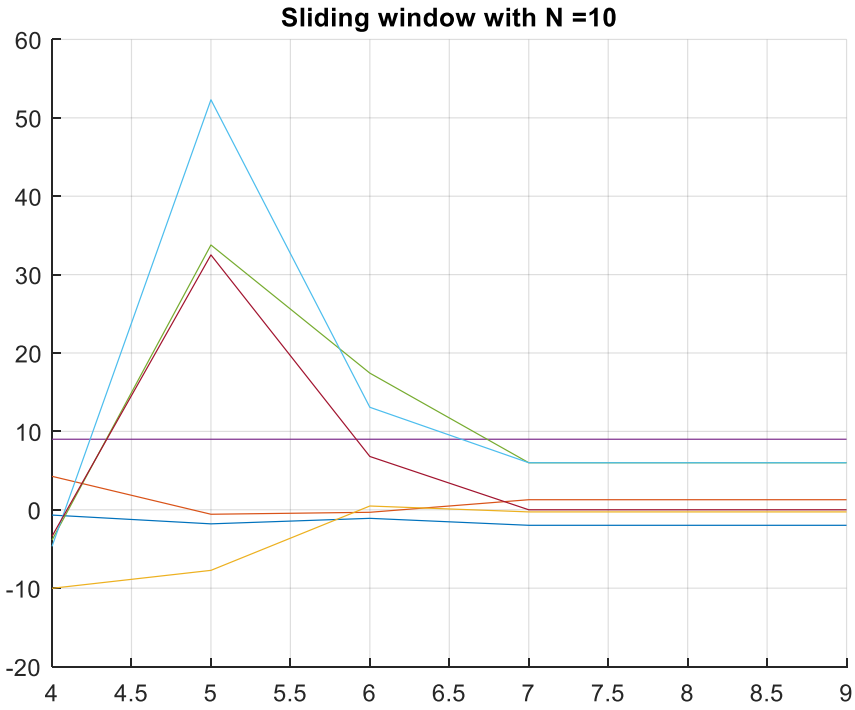
After completing the three RLS algorithms, I moved onto the sliding window algorithms. The first of these algorithms being the basic sliding window least squares. The way that the sliding window least squares algorithm estimates the parameters differs from the other algorithms as the way the window interacts with the simulation is quite different from the other algorithms. Due to many of these differences, I opted to use the recursive form instead of the standard form for calculating the sliding window least squares. This involves subtracting the value that is at the backend of the window as it shifts to erase it entirely from the estimation.

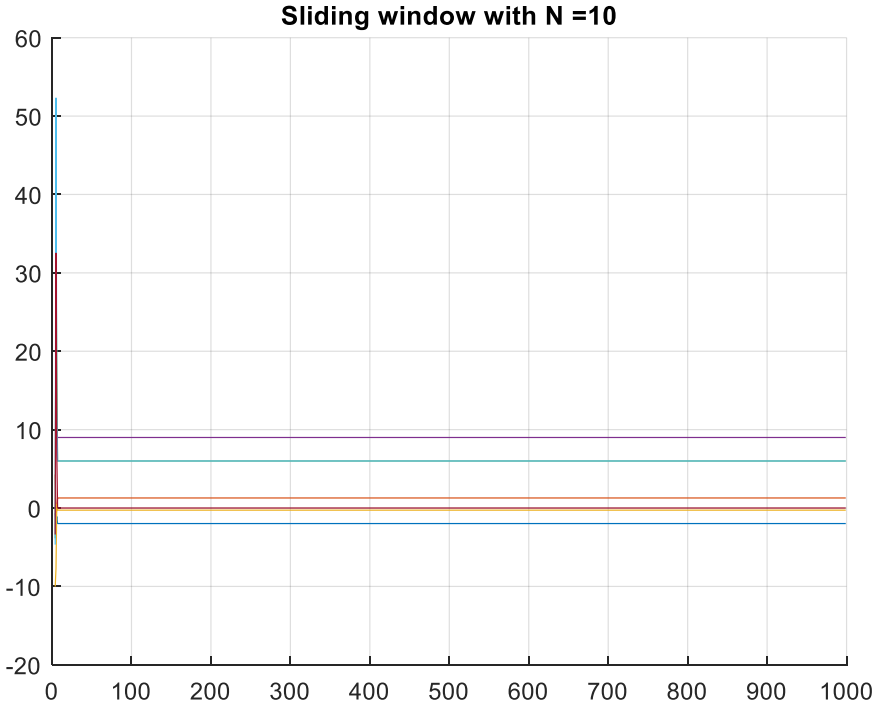


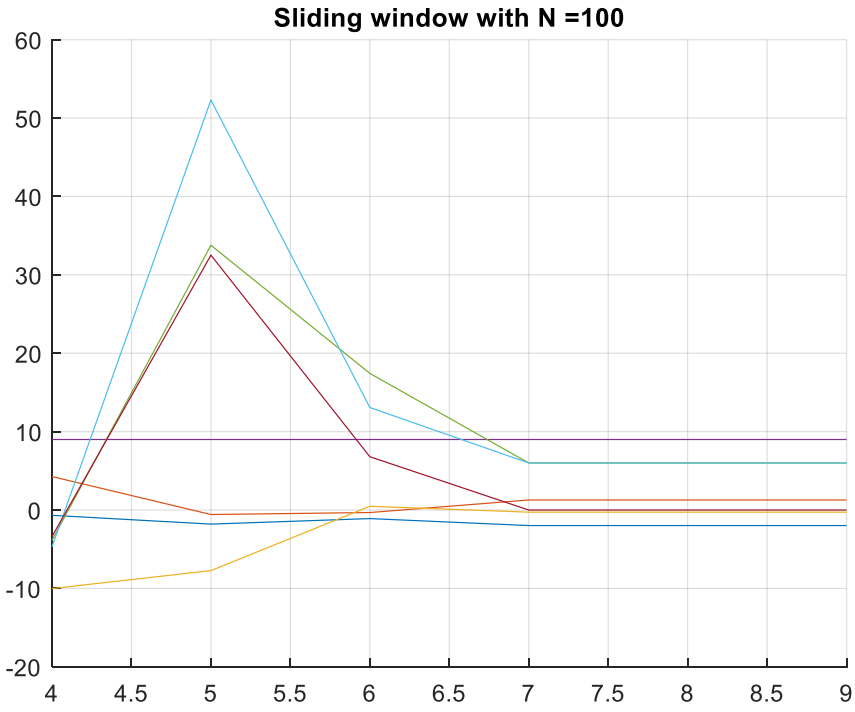
The function that implements the sliding window least squares algorithm lives in SlidingWindow.m and takes the system input, output, time, window size and a graph count as inputs for the function. The function then estimates the parameters of the system over time using the sliding window least squares algorithm and outputs a matrix of the seven parameters being estimated over time. The code for this algorithm is shown below.

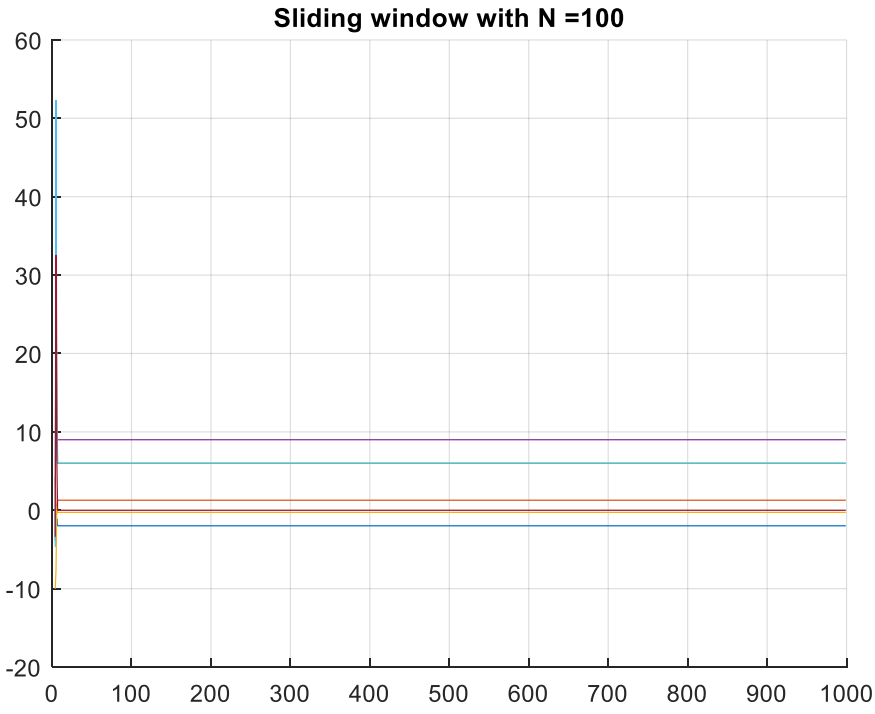
```
for i=1:numSamples
    %Create the phi matrix
    for j=1:3
        if(i-j <=0)
            phi(i,j)=0;
        else
            phi(i,j)=-output(i-j);
        end
    end
    for j=0:3
        if(i-j-1 <=0)
            phi(i,j+4)=0;
        else
            phi(i,j+4)=input(i-j-1);
        end
    end

    %Sliding window calculation
    if(i <= N)
        p = p + phi(i,:)' * output(i);
        q = q + phi(i,:)' * phi (i,:);
        theta(:,i) = inv(q) * p;
    else
        p = p + phi(i,:)'*output(i) - phi(i-N,:)'*output(i-N);
        q = q + phi(i,:)'*phi(i,:) - phi(i-N,:)'*phi(i-N,:);
        theta(:,i) = inv(q) * p;
    end
end
```

Below are charts for this function running with N = 10 and N = 100. The charts are shown with an x-axis of 10 and 1000 for better visibility.





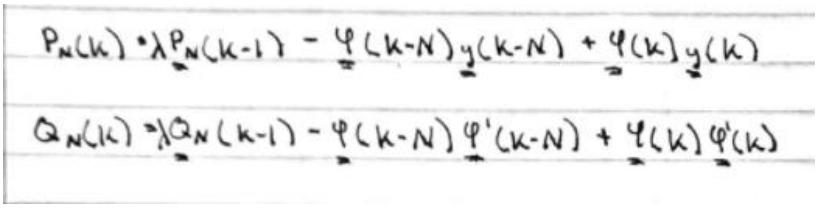




As shown above, both of the sliding window sizes quickly converge at the final estimates, then proceed to stay at the converged value as the simulation runs to completion.

Question 8: Sliding Window Least Squares with Forgetting Factor & Change at t=411

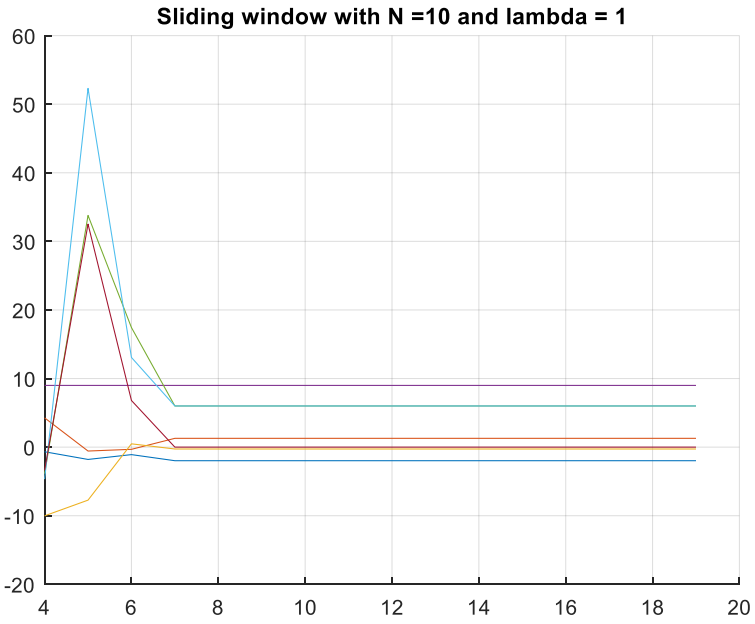
Once the first sliding window least squares algorithm was complete, I added the forgetting factor and a sudden change to the simulation. The sudden change occurs at t=411 and required some additional logic to allow for the window to 'forget' the sample at time – windowSize correctly. The change in the recursive form of the p and Q calculations is shown below.

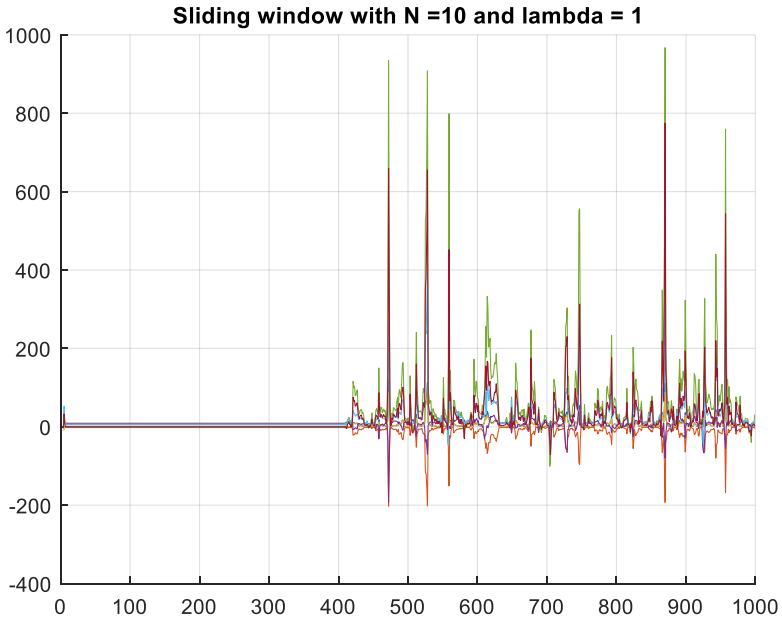


This function lives in SW_forgetting.m and is prone to noise in the parameter estimation which is a flaw I was unable to completely eliminate in the code. The code for the parameter estimation is shown below, as the remainder of the code is the same as the basic sliding window least squares algorithm.

```
% Generate theta with sliding window
% Note: This is inside of this if block to handle 'forgetting' the correct
% values from the original vs changed paramater outputs
if(i <= N)
    p = lambda(L) * p + phi(i,:)' * output(i);
    q = lambda(L) * q + phi(i,:)' * phi (i,:);
    theta{L}(:,i) = inv(q) * p;

elseif(i > N && i < changeParam)
    p = lambda(L) * p + phi(i,:)'*output(i) - phi(i-N,:)'*output(i-N);
    q = lambda(L) * q + phi(i,:)'*phi(i,:) - phi(i-N,:)'*phi(i-N,:);
    theta{L}(:,i) = inv(q) * p;

elseif(i >= changeParam && i <= changeParam + N)
    p = lambda(L) * p + phi(i,:)'*output2(i) - phi(i-N,:)'*output(i-N);
    q = lambda(L) * q + phi(i,:)'*phi(i,:) - phi(i-N,:)'*phi(i-N,:);
    theta{L}(:,i) = inv(q) * p;

else
    p = lambda(L) * p + phi(i,:)'*output2(i) - phi(i-N,:)'*output2(i-N);
    q = lambda(L) * q + phi(i,:)'*phi(i,:) - phi(i-N,:)'*phi(i-N,:);
    theta{L}(:,i) = inv(q) * p;
end
```

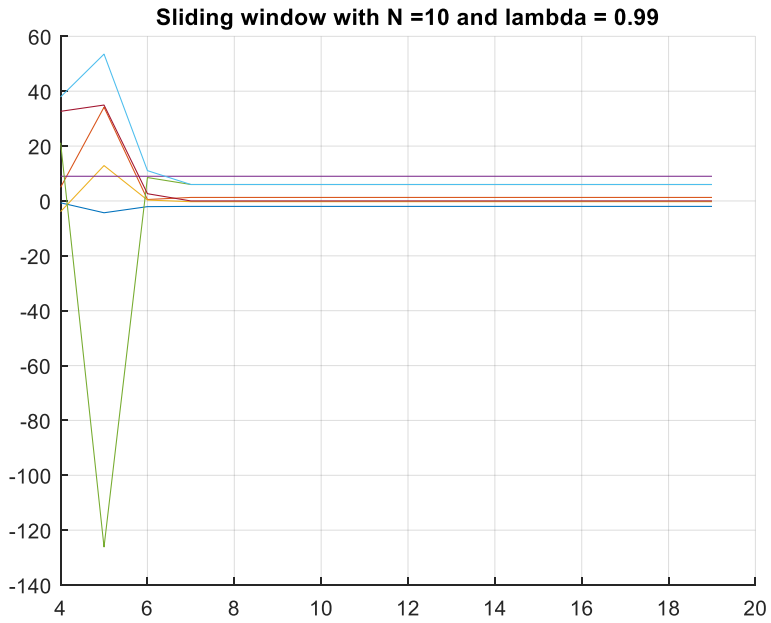
The reasoning for the if else blocks that cause different ways the parameters are estimated, and versions of the output was to handle how the sliding window subtracted the correct values during the sudden change in the simulation. Graphs from this function are shown below, with iterations for a window size of 10 and 100 as well as lambda values for 1, 0.9 and 0.99.

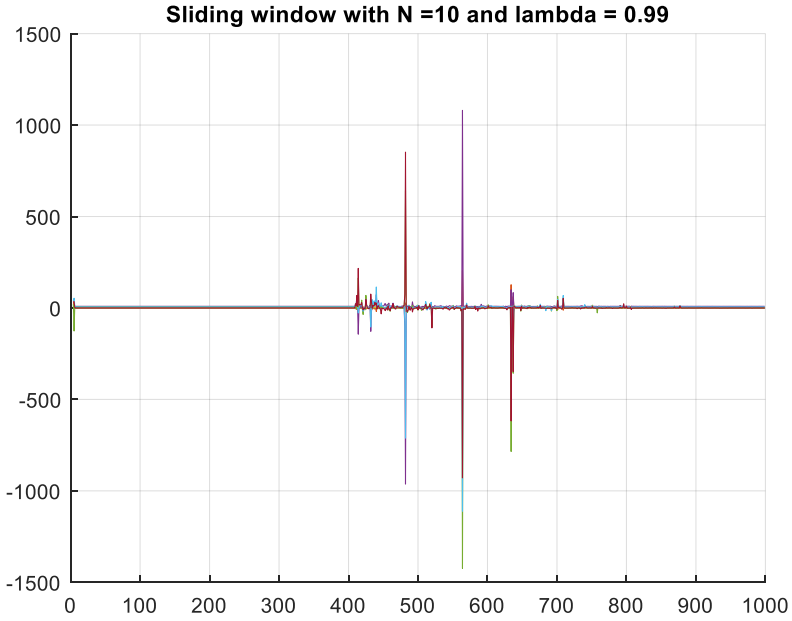




During this simulation, we can see from the first graph that the sliding window least squares estimate quickly converges to the correct lambda values, however with the sudden change and the very small window size and a lambda factor of 1 the system is too responsive to the sudden change and does not settle down from the sudden change at t=411. Due to this issue with the system, it does not settle during the 1000 samples of the simulation and does not finish at the correct lambda values after the sudden change. It is worth noting that all sliding window graphs use the same base logic for the change of parameters in the simulation and the only

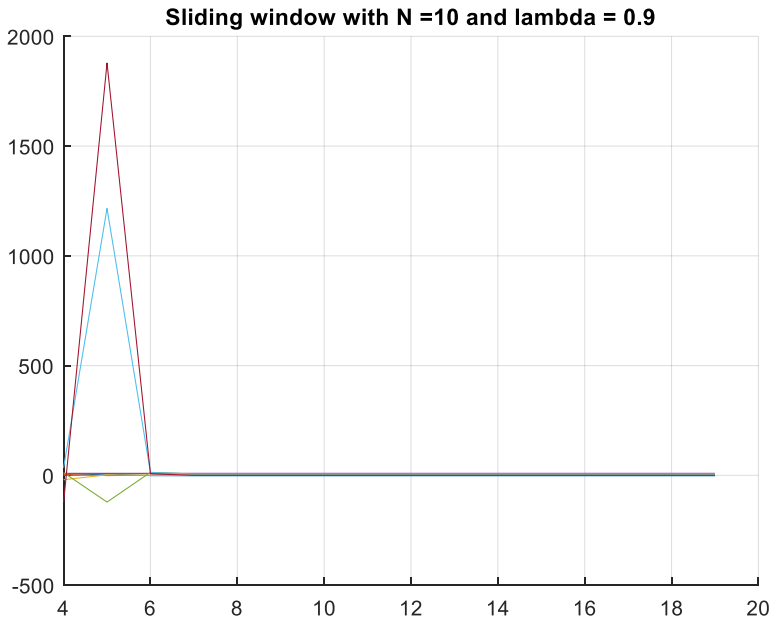
changes are the window size and the lambda values, leading me to believe that the behavior shown is an effect of the system design and not of a bad simulation.

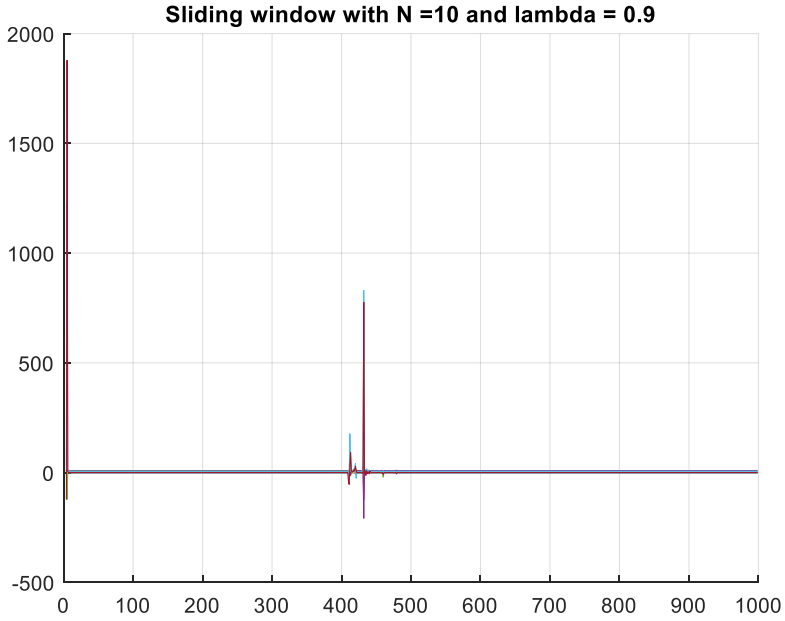




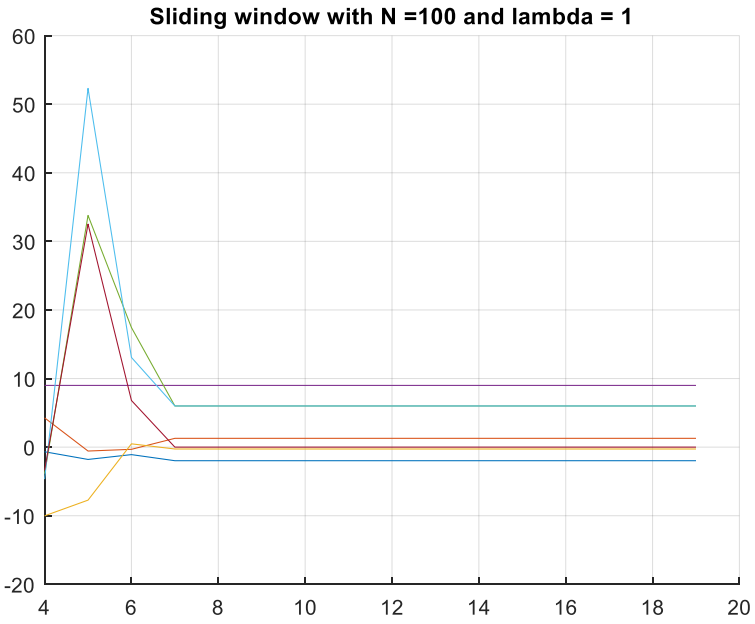
Next, we have the graphs for the simulation when the window size is 10 and we have a forgetting factor of 0.9. In this case we can again see the parameter estimation quickly converge to the correct parameters for the simulation in the first chart and then remain at the desired values. With the second chart we can see how the system react to the change at t=411 where it

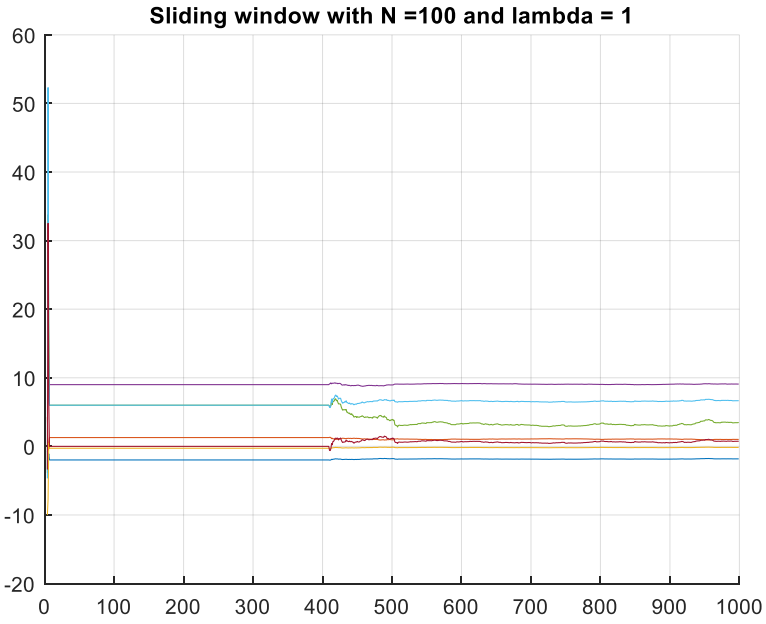
has a few spikes based on the new parameters, but then proceeds to return to a steady, correct estimation of the parameters.



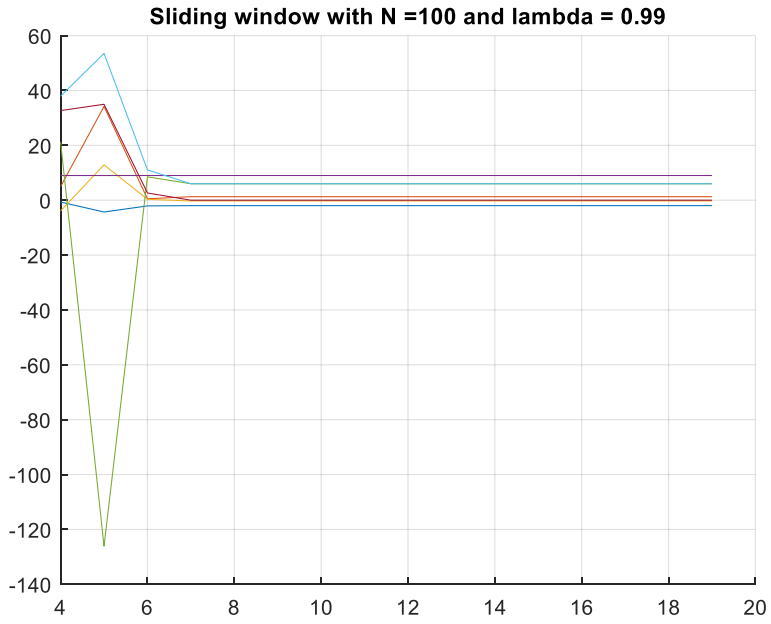


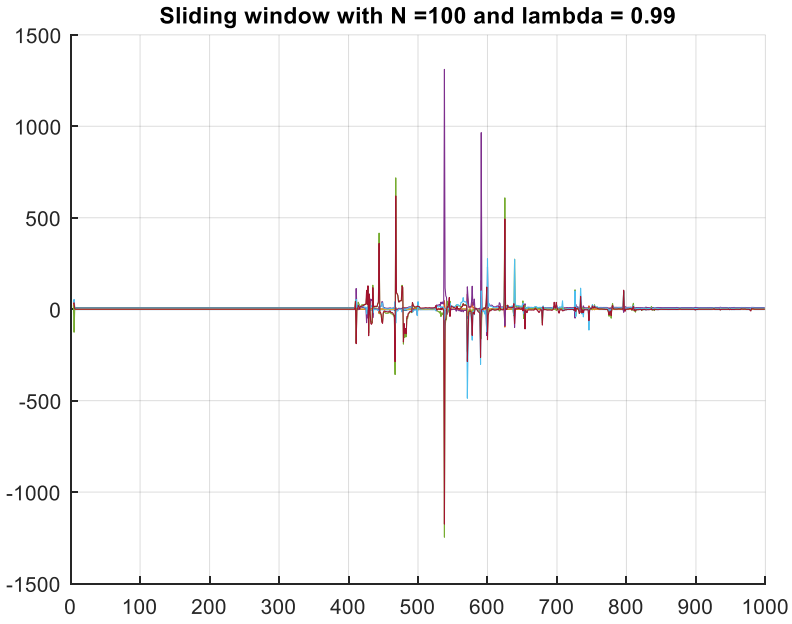
Finally, for the window size of 10 we can see once again the parameters quickly converge for the beginning of the simulation. At t=411, there is again a spike in the estimation, but it quickly re-converges onto the correct, changed, parameters for the estimation.



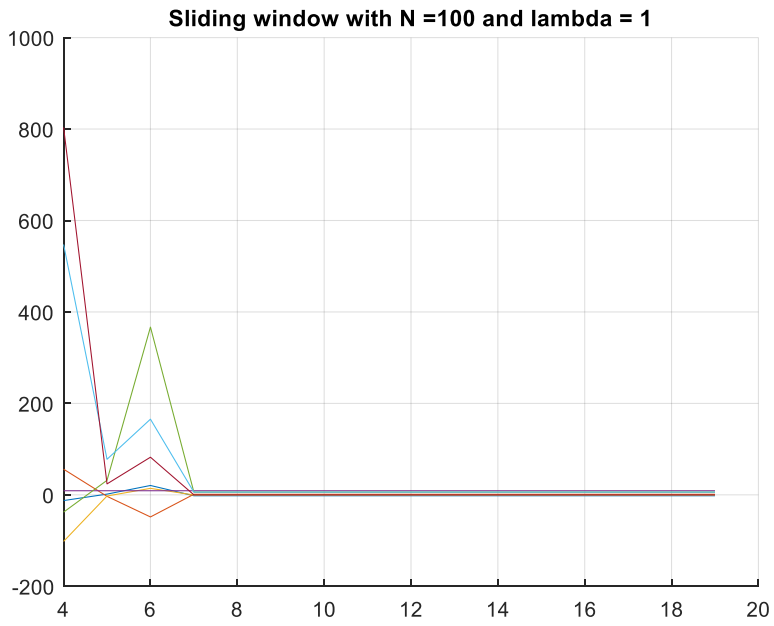


Next, we have the sliding window with a window size of 100. Again, we can see the quick convergence of the initial parameters and the parameter estimation holding steady until the simulation changes parameters. At which point, the simulation changes to the newly estimated parameters and holds near their exact value for the remainder of the simulation.





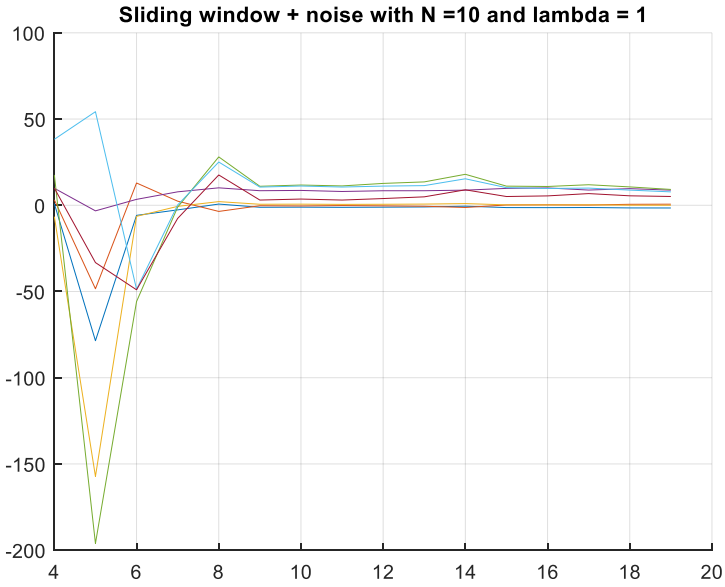
Next, we have the sliding window with a window size of 100 and a forgetting factor of 0.99. We can see the initial parameters converge quickly and remain at the simulation's correct parameters until the sudden change occurs. At which point, the simulation faces some large fluctuations in the estimated parameters, but these fluctuations eventually converge to the correct parameters once again.

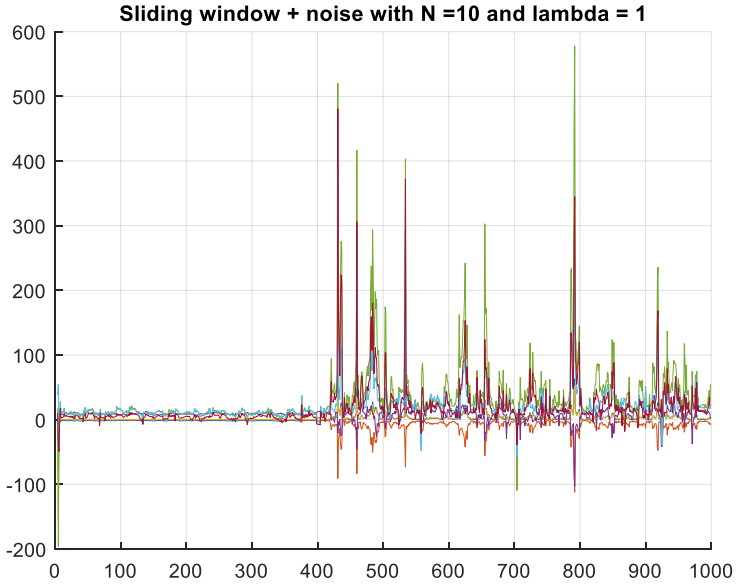


Finally, we have the sliding window with n=100 and lambda =1. As we can see from this simulation, the parameters quickly converge and remain at the converged parameter values until the change in parameters at t=411. There is some gradual change after the change in parameters, as the estimate shifts from the first set of parameters to the second set of parameters, but this is difficult to see with the initial spike of parameters.

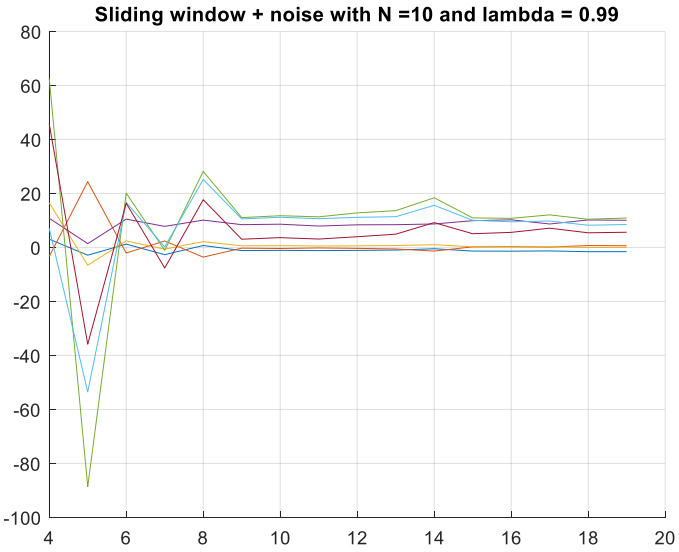
Sliding Window Least Squares with Forgetting Factor, Noise and Change at t=411:

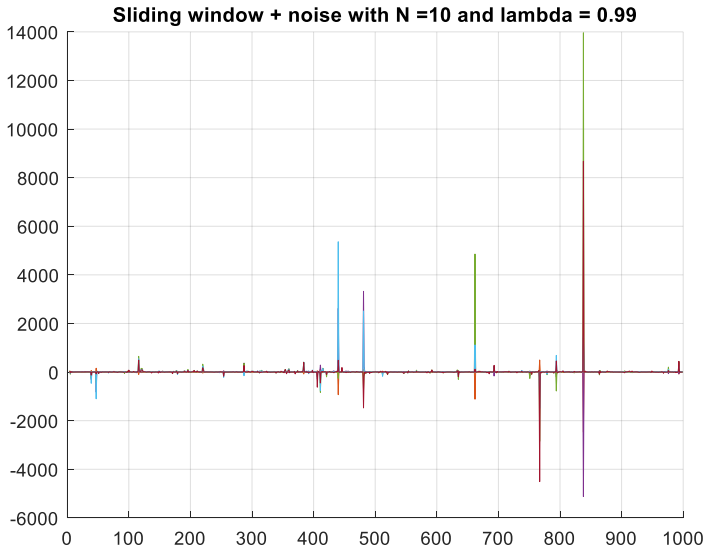
Finally, the sliding window least squares with forgetting factor, noise and a sudden change was the last algorithm to complete. The sudden change to the parameters of the simulation occurs again at t=411 and the noise is added to both the original and changed outputs of the system. The function lives in SW_noise and while it does have similar issues to the previously mentioned algorithm, it still estimates the parameters accurately. The code for this function is nearly identical to SW_forgetting.m as the only change is the inclusion of the white noise that is generated in the same manner as the recursive sliding window. Graphs from this function are shown below.

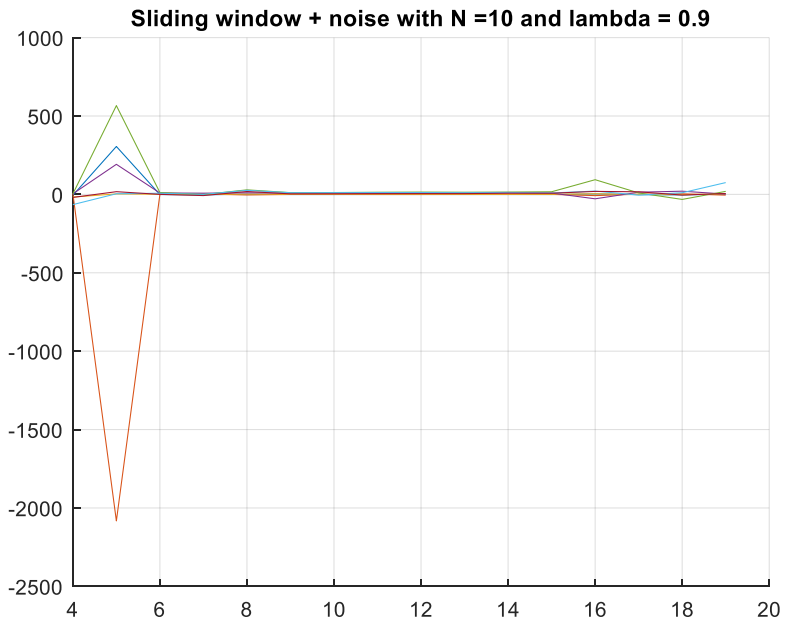


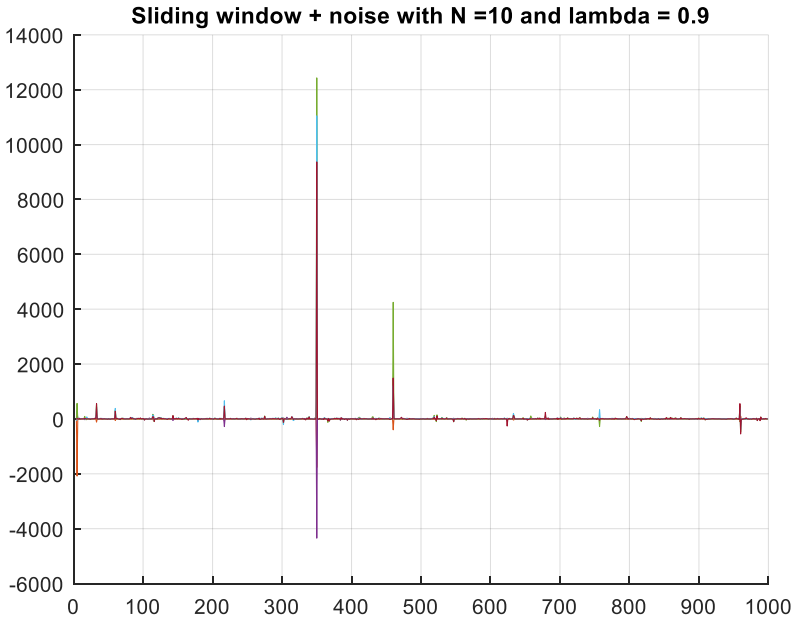


During this simulation, we can see from the first graph that the sliding window least squares estimate quickly converges to the correct lambda values, however with the sudden change, output noise, and the very small window size and a lambda factor of 1 the system is too responsive to the sudden change and does not settle down from the sudden change at t=411. Due to the system's very high response to noise and change, it does not settle during the 1000 samples of the simulation and does not finish at the correct lambda values after the sudden change. It is worth noting that all sliding window graphs use the same base logic for the change of parameters in the simulation and the only changes are the window size and the lambda values, leading me to believe that the behavior shown is an effect of the system design and not of a bad simulation.

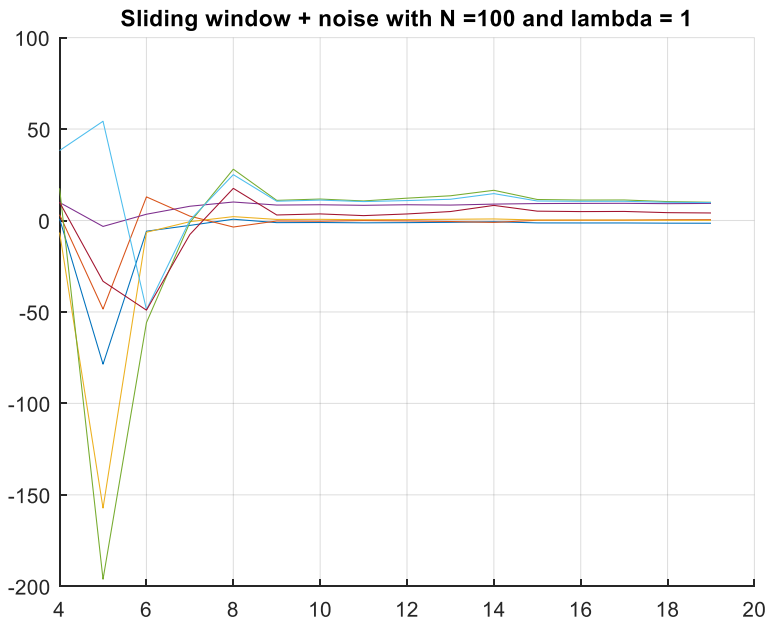


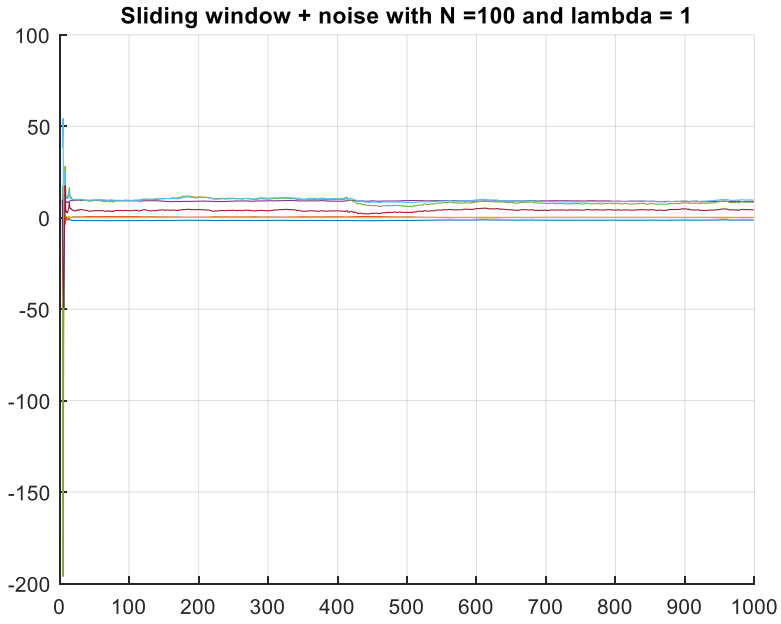


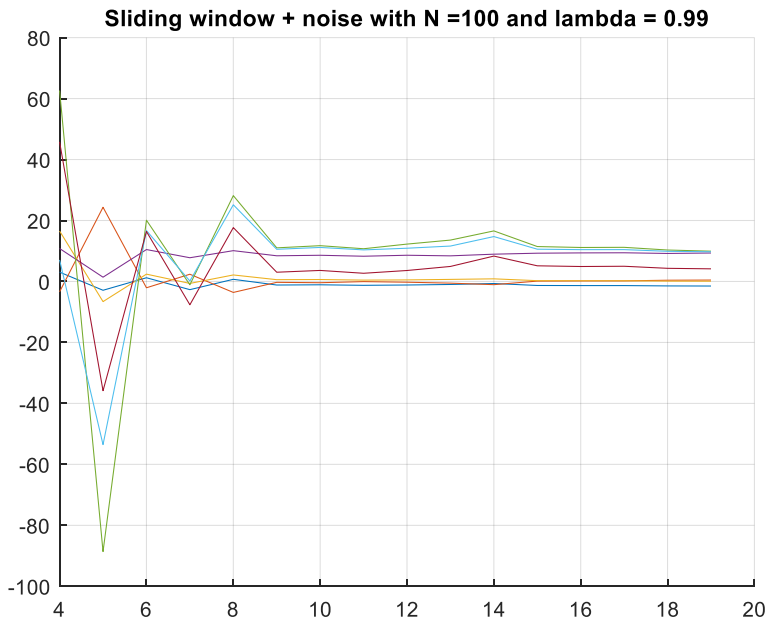


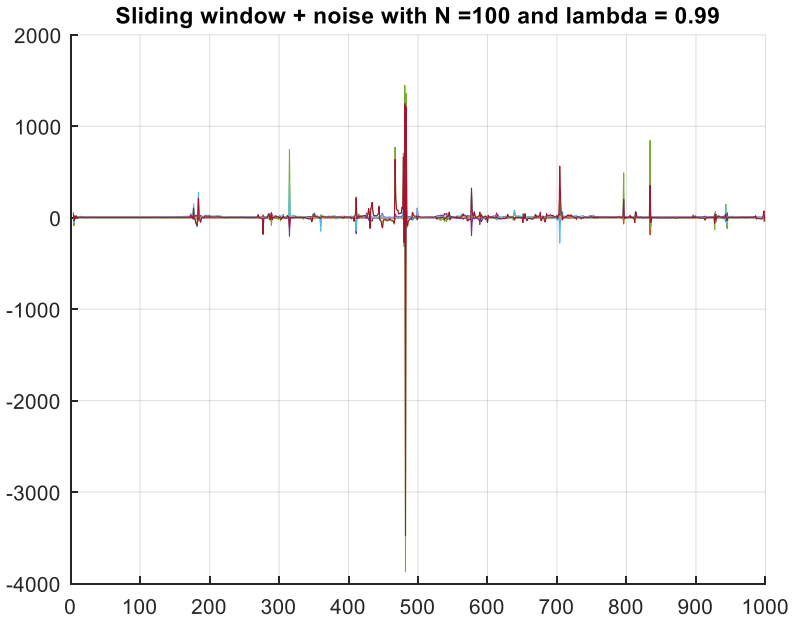


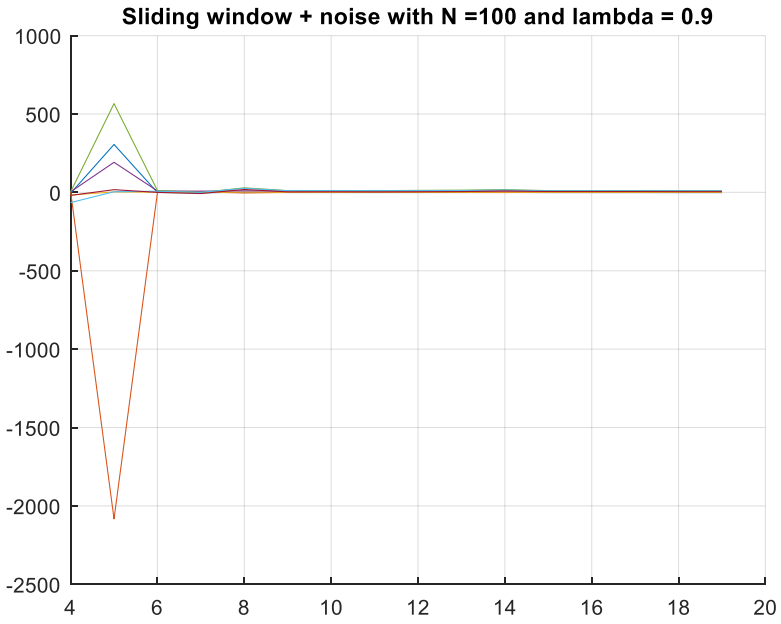
For the graphs with lambda values of 0.9 and 0.99 with a window size of 10 we can see on both how the initial parameters again quickly converge despite the noise. There are some small spikes in the estimation of the parameters due how the system is designed but the parameters maintain the ability to converge on both the initial set of parameters and adjust to the second of parameters after the sudden change at t=411.

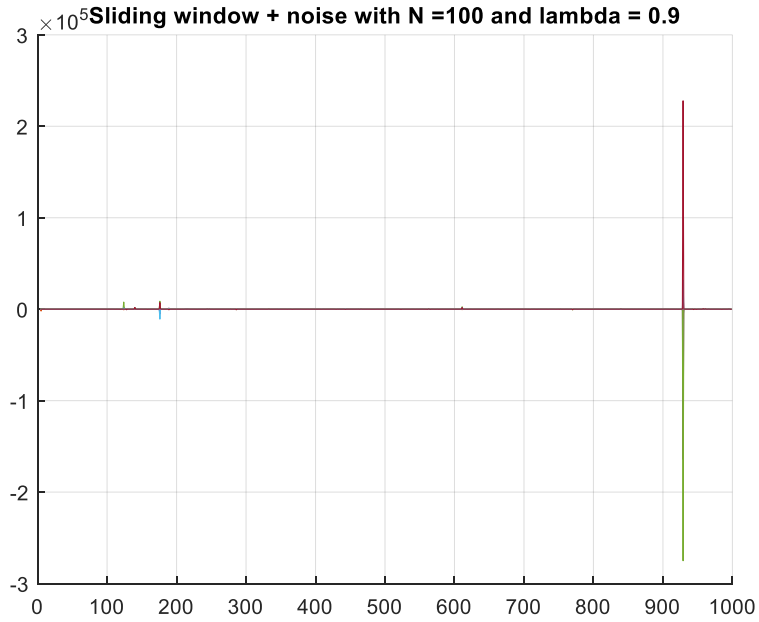

Sliding window + noise with N =100 and lambda = 1
100
50
0
-50
-100
-150
-200
4
6
8
10
12
14
16
18
20


Sliding window + noise with N =100 and lambda = 1
100
50
0
-50
-100
-150
-200
0
100
200
300
400
500
600
700
800
900
1000


Sliding window + noise with N =100 and lambda = 0.99
80
60
40
20
0
-20
-40
-60
-80
-100
4
6
8
10
12
14
16
18
20


Sliding window + noise with N =100 and lambda = 0.99
2000
1000
0
-1000
-2000
-3000
-4000
0
100
200
300
400
500
600
700
800
900
1000





For the remaining graphs with a sliding window size of 100 and the three lambda values of 1, 0.99 and 0.9 we can once again see the parameters quickly converge on the parameter's values after just a few samples. These systems are more resilient to the noise than the sliding windows with a window size of 10 and as such are able to accurately estimate the parameters of the system. These systems do still experience some spikes in the estimations of the parameters but are able to still re-converge after these spikes and after these sudden change of parameters in these systems.

Conclusions

Overall, most of the algorithms presented above are able to quickly identify the parameters and adapt to the sudden change of parameters and noise added to the output. The sliding window algorithm was less resilient to the noise, especially with a large lambda and a short window size. However, these systems were very adaptive to the changes in the system. On the other side, the recursive algorithms were far more stable in their estimations over time and less adaptive to change especially with a large lambda. These different systems have vastly different characteristics even though many of their base principles are the same the behavior shown in the simulations above and the other simulations run throughout this project show. These differences each hold value based on their use case and I believe all of these algorithms have their uses given different systems.